

Yitae Jeong

LLM / RAG / Inference Engineer

jeongiitae6@gmail.com

TECHNICAL PROFILE

LLM engineer focused on both on-prem LLM inference and API-based Graph-RAG systems. Designed and benchmarked SEOCHO, an open source Graph-RAG runtime that combines document ingestion, semantic extraction, vector/graph retrieval, evidence contracts, answer synthesis, and tracing. Comfortable moving from idea to experiment to product feature: define the failure mode, measure it, then turn the result into a runtime contract or guardrail. Recent infrastructure work includes vLLM + LMCache, Blackwell Pro-class server setup, RDMA/NIXL/UCX/RoCEv2 networking, KV-cache offloading, and TTFT-focused workload analysis.

SELECTED ENGINEERING RESULTS

On-Prem Inference / Multi-Node Serving

- Built an on-prem LLM serving foundation around Blackwell Pro-class hardware, vLLM, LMCache, and KV-cache workload analysis, with focus on TTFT, throughput, prefill/decode balance, and cache-tier design.
- Worked on multi-node serving prerequisites: RDMA setup, NIXL, UCX, RoCEv2, and NIC-engine tuning for high-throughput KV movement and cache offloading.
- Problem: long-context inference wastes TTFT on repeated prefill. Worked on vLLM + LMCache KV offloading and reuse-vs-recompute decisions to reduce recomputation pressure and improve first-token latency.
- Prepared KCC2026 tutorial material on inference workloads, KV-cache footprint, vLLM internals, LMCache data paths, scheduler-side decisions, and worker-side KV movement.

API-Based Graph-RAG / Product Runtime

- Problem: graph retrieval was assumed to help financial QA. Built a FinDER-style study on **910 financial QA rows** across six slices to compare vector, graph, and vector+graph modes with ontology size as a controlled variable.
- Problem: decision-thread QA looked graph-friendly. Ran **135 cases x 5 lanes = 675 comparison runs** on BC3 and found vector/hybrid remained stronger than graph-only, changing the product direction from "graph always wins" to route-by-query and evidence-first answering.
- Problem: graph context can be larger and expensive. Measured KV-cache cost behavior on BC3: graph context reached a **59% cached-token hit ratio** vs vector **0%**, with lower amortized billable prompt tokens despite **2.2x** larger raw context.
- Problem: final answers could hide missing graph evidence. Added route and evidence contracts so required slots, missing slots, provenance, route profile, and answer shape remain visible before answer synthesis.
- Converted API-model limitations into product design: ontology guardrails, context graph, route profiles, and evidence bundles compensate for not owning the foundation model while keeping outputs auditable.
- Profiled DozerDB/Neo4j graph hydration and adopted a Rust PackStream codec: **1.62x** faster real 3-shard federation read and **3.57x** faster 130k-row bulk hydration with hash-parity checks.

ENGINEERING PROJECTS

SEOCHO: Ontology-Grounded Graph-RAG and Agent Runtime

Open source, 2026-present | GitHub: github.com/tteon/seocho | DeepWiki: deepwiki.com/tteon/seocho

- Implemented the API-side ingestion-to-answer path: raw records, chunking, entity extraction, graph writes, semantic retrieval, and answer synthesis.
- Benchmarked vector, graph, and hybrid retrieval lanes with fixed answerer controls, paired comparisons, and persisted JSONL/Opik traces.
- Converted experiment findings into product contracts: ontology profile selection, evidence bundles, route profiles, answer-shape metadata, and missing-slot guardrails.
- Prepared KGC2026 material using SEOCHO to explain graph-agentic RAG across RDF/LPG specialist agents, ontology guardrails, and evidence-grounded answers.
- Used ontology guardrails and context-graph design to improve reliability when the system depends on external foundation-model APIs instead of owning the model weights.
- Implemented layered memory graph contracts such as Document -> DocumentVersion -> Section -> Chunk -> Entity, with vector metadata joinable to graph provenance.
- Built semantic routing across LPG/RDF graph-specialist paths, answer generation, and graph reasoning guardrails.
- Added evidence-bundle fields for slot fills, missing slots, provenance, route profile, answer shape, and conservative abstention.

On-Prem LLM Inference Infrastructure: vLLM, LMCache, RDMA/NIXL

Current work, 2026-present | Multi-node serving infrastructure and workload analysis

- Set up server infrastructure for LLM inference workloads, including Blackwell Pro-class hardware preparation and software stack bring-up.
- Built a vLLM + LMCache serving stack to study KV-cache reuse, prefix caching, chunked prefill, scheduler behavior, and cache-tier design.
- Worked on multi-node serving requirements including RDMA setup, NIXL, UCX, RoCEv2, and NIC-engine tuning for KV movement and cache offload paths.
- Analyzed workload dimensions including ISL, OSL, prefill/decode ratio, TTFT, ITL/TPOT, E2E latency, throughput, cache hit ratio, and GPU memory pressure.

- Evaluated KV offloading and reuse-vs-recompute tradeoffs using KV size, transfer bandwidth, cache-tier latency, prompt length, and workload locality.
- Prepared KCC2026 tutorial material on inference workload, KV-cache behavior, vLLM internals, and LMCache integration.
- Used workload analysis to reason about multi-GPU serving design, especially GPU memory pressure, KV locality, tiered cache sizing, and prefill-heavy workload optimization.

TECHNICAL SKILLS

LLM/RAG: Graph-RAG, vector retrieval, hybrid retrieval, grounded prompting, tool calling	Agents: semantic routing, graph-specialist agents, evidence contracts, answer guardrails
Evaluation: LLM-as-judge, fixed-answerer comparisons, paired tests, benchmark design	Inference: vLLM, LMCache, KV offload, prefill/decode, TTFT, ITL/TPOT, throughput
Infrastructure: RDMA, NIXL, UCX, RoCEv2, GPU server setup, cache tiering, observability	Stack: Python, FastAPI, OpenAI Agents SDK, DozerDB/Neo4j, Opik, JSONL tracing

SELECTED LINKS

SEOCHO GitHub: <https://github.com/tteon/seoch0>
SEOCHO DeepWiki: <https://deepwiki.com/tteon/seoch0>